PRACTICAL 2	Implementation and Time analysis of linear and binary search algorithm

Objective

- To implement **Linear Search** and **Binary Search** algorithms in C.
- To compare their execution time and efficiency.
- To analyze the both searching techniques

Linear search.

Algorithm

1. Start from the first element of the array.
2. Compare each element with the key.
3. If the key is found, return its position.
4. If the end of the array is reached without finding the key, report that the element is not found.

C Program

```
#include <stdio.h>


int main() {
   int n, i, key;


   printf("Enter number of elements: ");
   scanf("%d", &n);


   int a[n];
```

```
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++) {
        if(a[i] == key) {
            printf("Element found at position %d", i + 1);
            return 0;
        }
    }

    printf("Element not found");
    return 0;
}
```

Sample Output

Enter number of elements: 5

Enter elements:

12 25 18 40 55

Enter element to search: 40

Element found at position 4

analysis

- Checks elements one by one.
- Works on both sorted and unsorted arrays.
- Efficient only for small datasets.

Conclusion

Linear Search is simple to implement and suitable for small or unsorted datasets. However, its performance decreases as the number of elements increases because it may need to examine every element.

Binary search

Algorithm

Note: The array must be sorted.

1. Set low = 0 and high = n-1.
2. Find the middle element.
3. If the middle element equals the key, return its position.
4. If the key is smaller, search the left half.
5. If the key is larger, search the right half.
6. Repeat until the element is found or the search interval becomes empty.

C Program

```
#include <stdio.h>


int main() {
   int n, i, key;
```

```
printf("Enter number of elements: ");
scanf("%d", &n);

int a[n];

printf("Enter sorted elements:\n");
for(i = 0; i < n; i++)
   scanf("%d", &a[i]);

printf("Enter element to search: ");
scanf("%d", &key);

int low = 0, high = n - 1, mid;

while(low <= high) {
   mid = (low + high) / 2;

   if(a[mid] == key) {
      printf("Element found at position %d", mid + 1);
      return 0;
   }
   else if(key < a[mid])
      high = mid - 1;
   else
      low = mid + 1;
```

```
    }

    printf("Element not found");
    return 0;
}
```

Sample Output

Enter number of elements: 6

Enter sorted elements:

10 20 30 40 50 60

Enter element to search: 50

Element found at position 5

Analysis

- Divides the search space into two halves after each comparison.
- Requires the array to be sorted.
- Much faster than Linear Search for large datasets.

Conclusion

Binary Search is highly efficient for searching in sorted arrays because it reduces the search space by half in every step. Its logarithmic time complexity makes it suitable for large datasets.